

HERMES AGENT | CLASE 03

Cómo montar y probar Hermes avanzado

Profiles, memoria externa, OpenViking, Hindsight, Codex, subagentes, Kanban, Cron y A2A

Bloque 1 - Profiles y sesiones separadas

Los Profiles son instancias independientes de Hermes dentro de la misma instalación. Cada Profile posee su propia configuración, sesiones, memoria, skills, cron jobs y estado.

El acceso remoto y los múltiples usuarios autenticados se tratarán en la Clase 4, junto con la creación y el uso de Skills y la integración con MCP.

Recomendación para esta clase: utiliza la línea de comandos para iniciar cada Profile. De esta forma, Hermes recibe explícitamente el Profile en el momento en que se crea el proceso y se reduce el riesgo de que una sesión herede estado, directorio de trabajo o contexto de otro Profile.

Desktop y Dashboard siguen siendo útiles para administración y visualización, pero no serán la vía principal para abrir las sesiones de los Profiles en esta clase. Para iniciar una sesión, utiliza preferentemente:

```
hermes -p <profile> chat
```

Antes de comenzar - si Hermes Desktop aparece pero no carga en WSL2

En WSL2, Hermes Desktop depende de la integración gráfica de Linux con Windows mediante WSLg. En algunos equipos, la ventana puede aparecer en la barra de tareas o mostrar una advertencia, pero la interfaz no cargar correctamente.

Primero, finaliza posibles procesos bloqueados en el terminal de Ubuntu:

```
pkill -f Hermes  
pkill -f electron  
pkill -f chromium
```

Después cierra Ubuntu. En Windows PowerShell, ejecuta:

```
wsl --shutdown
```

Abre Ubuntu nuevamente e intenta iniciar Desktop:

```
cd ~/hermes-lab  
hermes desktop
```

Este procedimiento reinicia el entorno gráfico y los procesos de WSL2 y es diferente del problema de cwd/Profile descrito más adelante en esta clase. Si Desktop sigue sin cargar, la demostración puede continuar por CLI o Dashboard mientras se diagnostica el problema gráfico.

1.1 Crear el Profile aluno

Abre Hermes Dashboard.

Accede a **Profiles**.

Selecciona **Create Profile**.

Crea un Profile con el nombre aluno.

Añade una descripción breve para identificar el Profile.

1.2 Configurar el Profile aluno

Si el Profile todavía no tiene un modelo/provider configurado, ejecuta el setup:

```
hermes -p aluno setup
```

Después del setup, inicia el Profile aluno directamente desde el terminal:

```
hermes -p aluno chat
```

1.3 Iniciar Profiles desde la línea de comandos

Siempre que abras o cambies de Profile durante la demostración, finaliza la sesión actual e inicia explícitamente el Profile deseado desde el terminal.

```
hermes -p aluno chat
```

```
hermes -p default chat
```

1.4 Confirmar sesiones separadas

Inicia una nueva sesión del Profile aluno con `hermes -p aluno chat`.

Envía un mensaje sencillo para identificar esa sesión.

Finaliza la sesión actual e inicia el Profile principal con `hermes -p default chat`.

Confirma que la sesión creada en el Profile aluno no aparece en el Profile principal.

Vuelve al Profile aluno con `hermes -p aluno chat` y confirma que la sesión anterior sigue disponible.

1.5 Prestar atención al cwd en Hermes Desktop

Hermes Desktop puede conservar un directorio de trabajo (cwd) antiguo entre sesiones o cambios de Profile. En esta situación, cambiar `terminal.cwd` en `config.yaml` puede no ser suficiente porque Desktop también mantiene el valor recordado en `hermes.desktop.workspace-cwd`.

Consecuencia: una nueva sesión puede abrirse en el Workspace anterior y cargar archivos de contexto, skills, memoria o configuraciones asociadas al proyecto equivocado. Por eso, la línea de comandos es el camino recomendado para la demostración de Profiles.

Corrección en Desktop cuando aparece este problema: cierra Hermes Desktop, confirma el `terminal.cwd` correcto en el `config.yaml` del Profile y elimina la clave `hermes.desktop.workspace-cwd` del almacenamiento local de Electron. En Windows, este almacenamiento se encuentra en

%APPDATA%\Hermes\Local Storage\leveldb\). Después abre Desktop nuevamente y crea una nueva sesión.

Este procedimiento es específico de Desktop. Los problemas de Profile/cwd en Dashboard o en gateways de larga duración tienen otro origen y no deben corregirse eliminando el LevelDB de Desktop.

Bloque 2 - Memoria persistente de Hermes

La memoria integrada de Hermes conserva información útil entre sesiones y está separada por Profile. Utiliza MEMORY.md y USER.md internamente, pero la demostración puede realizarse directamente desde el chat.

2.1 Demostrar la memoria entre sesiones

En el Profile alumno, inicia una nueva conversación.

Informa una preferencia sencilla, por ejemplo: “Prefiero ejemplos en Python cuando haya código”.

Finaliza la sesión y abre otra sesión en el mismo Profile.

Pregunta qué lenguaje debe priorizarse en los ejemplos.

Bloque 3 - Memoria externa: concepto antes de la instalación

Hermes posee memoria interna y también permite conectar providers de memoria externa. La memoria externa mantiene conocimiento persistente más allá de la memoria integrada y puede recuperar únicamente la información relevante para cada pregunta.

La memoria interna sigue activa junto con el provider externo. Sin embargo, cada Profile puede tener solo un provider externo activo a la vez.

3.1 Por qué la memoria externa puede reducir el uso de contexto

Cuando se colocan documentos grandes directamente en la conversación, parte o todo su contenido pasa a ocupar la ventana de contexto del modelo. Con recuperación bajo demanda, el sistema busca primero la información relevante y envía al modelo solo el fragmento necesario.

Esto no aumenta la ventana máxima de contexto del modelo. La ventaja está en evitar que grandes volúmenes de información permanezcan en el contexto en cada interacción, lo que puede reducir el consumo de tokens de contexto.

3.2 OpenViking y Hindsight resuelven problemas diferentes

OpenViking: organiza el conocimiento en una jerarquía de recursos y utiliza carga por capas. Es especialmente útil para recuperar partes específicas de documentos sin cargar todo de una vez.

Hindsight: trabaja con memoria a largo plazo, entidades, relaciones y síntesis/reflexión sobre hechos almacenados. Su principal ventaja es estructurar y relacionar conocimiento; el ahorro de contexto puede producirse mediante recuperación bajo demanda, pero no es su función principal.

Como solo un provider externo puede estar activo por Profile, el Profile alumno se utilizará con OpenViking y el Profile lois con Hindsight.

Bloque 4 - OpenViking como memoria externa

4.1 Instalar uv en Ubuntu/WSL2

Instala primero uv mediante el instalador oficial de Astral. Ejecuta en el terminal de Ubuntu/WSL2:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version
```

Si uv --version muestra la versión instalada, continúa. Si el comando todavía no se encuentra, cierra y vuelve a abrir el terminal de Ubuntu y prueba nuevamente.

Evita utilizar sudo snap install astral-uv --classic como ruta principal. El instalador independiente de Astral evita la advertencia de classic confinement de Snap y sigue la documentación oficial de uv.

4.2 Instalar y preparar OpenViking

Elige la forma de instalación según el tipo de embedding que se utilizará en OpenViking.

Opción 1 - Instalación estándar

Utiliza esta opción cuando los embeddings se configurarán mediante una Cloud API o mediante Ollama.

```
uv tool install openviking --upgrade
```

Opción 2 - Embedding ligero en CPU

Utiliza esta opción cuando quieras seleccionar **Lightweight CPU embedding** en el asistente sin depender de Ollama ni de una API externa para los embeddings.

```
uv tool install "openviking[local-embed]" --upgrade
```

El extra local-embed añade la dependencia necesaria para embeddings locales basados en llama.cpp.

Si OpenViking ya fue instalado con la Opción 1 y después decides utilizar Lightweight CPU embedding, ejecuta el comando de la Opción 2 para actualizar la instalación con ese extra.

Después de la instalación, inicia el asistente de configuración:

```
openviking-server init
```

En el asistente, elige **Step-by-step setup**. En **Embedding setup**, selecciona **Cloud API**, **Local via Ollama** o **Lightweight CPU embedding** según la opción de instalación elegida anteriormente.

En el paso VLM, elige el provider que se utilizará para interpretación y síntesis. Un flujo probado es **OpenAI Codex (Subscription)**, que utiliza autenticación de la suscripción; pueden utilizarse otras opciones según el entorno.

Si Lightweight CPU embedding informa que llama-cpp-python no está instalado, no continúes con una configuración incompleta. Sal del asistente y ejecuta:

```
uv tool install "openviking[local-embed]" --upgrade
```

Después ejecuta nuevamente openviking-server init.

Tras completar el asistente, valida la configuración:

```
openviking-server doctor
```

4.3 Utilizar el servidor local sin API key

Cuando OpenViking se ejecuta únicamente en localhost, con host 127.0.0.1, no es necesaria una API key. Sin root_api_key configurada, el servidor entra automáticamente en modo de desarrollo (dev mode), con autenticación deshabilitada.

Configuración local mínima en ~/.openviking/ov.conf:

```
{
  "server": {
    "host": "127.0.0.1",
    "port": 1933
  }
}
```

No configures root_api_key como una cadena vacía. Para uso local, elimina el campo. Si el servidor se expone a la red o a otro equipo, configura autenticación con una root_api_key antes de cambiar el host a una dirección no local.

4.4 Iniciar el servidor OpenViking

```
openviking-server
```

De forma predeterminada, el servidor utiliza la configuración en ~/.openviking/ov.conf. En la primera configuración, init prepara el archivo y doctor comprueba si el entorno local es utilizable.

Mantén este terminal abierto mientras Hermes esté utilizando OpenViking. El servidor normalmente se inicia en el puerto 1933.

4.5 Activar OpenViking local en el Profile alumno

Abre un segundo terminal de Ubuntu. Mantén abierto el terminal de openviking-server.

```
hermes -p alumno config set memory.provider openviking
```

Configura el endpoint local de OpenViking en el archivo .env del Profile alumno:

```
echo "OPENVIKING_ENDPOINT=http://127.0.0.1:1933" >> ~/.hermes/profiles/aluno/.env
```

El asistente hermes -p alumno memory setup puede exigir una API key incluso cuando el servidor local está en dev mode. Para uso local sin autenticación, configura el provider y el endpoint manualmente con los comandos anteriores.

```
hermes -p alumno memory status
```

Utiliza memory status para confirmar que OpenViking está activo en el Profile alumno.

Importante: cada Profile puede utilizar su propia configuración de memoria externa. En esta clase, el Profile alumno se utilizará con OpenViking y el Profile lois con Hindsight.

La memoria persistente nativa de Hermes sigue existiendo; el provider externo añade una capa especializada de almacenamiento y recuperación.

4.6 Añadir y consultar conocimiento

Inicia una nueva sesión del Profile alumno con hermes -p alumno chat.

Añade un recurso pequeño a OpenViking utilizando las herramientas expuestas por el provider.

Haz una pregunta específica sobre el contenido añadido.

Pide a Hermes que indique el recurso o URI consultado.

Bloque 5 - Hindsight y memoria a largo plazo

Hindsight se utilizará en el Profile lois porque el Profile alumno ya tiene OpenViking como provider externo activo.

5.1 Preparar el Profile lois

Si el Profile lois ya existe, no es necesario crearlo nuevamente. Si todavía no existe, créalo antes de continuar.

Si lois aún no tiene modelo/provider configurado, ejecuta:

```
hermes -p lois setup
```

5.2 Instalar Hindsight Local Embedded manualmente

Para evitar el timeout del instalador automático, instala previamente el paquete completo de Local Embedded en el entorno Python de Hermes:

```
uv pip install --python ~/.hermes/hermes-agent/venv/bin/python hindsight-all
```

El asistente de Hermes puede intentar instalar hindsight-all automáticamente. En algunos entornos, esta instalación supera el límite de 120 segundos y muestra uv pip install timed out. La instalación manual anterior evita depender de ese timeout.

5.3 Configurar Hindsight en el Profile lois

Ejecuta:

```
hermes -p lois memory setup
```

En el menú, selecciona **Hindsight** y después **Local Embedded - Run Hindsight locally**.

Cuando se solicite, elige el provider de LLM. El flujo probado utiliza MiniMax. En el modo Local Embedded, los embeddings y el reranking se ejecutan localmente.

Si el asistente todavía intenta instalar hindsight-all y muestra timeout, completa o cancela esa etapa, vuelve a ejecutar el comando manual de instalación y repite `hermes -p lois memory setup`.

5.4 GPUs antiguas o incompatibilidad CUDA: forzar CPU

Si `retain` o `recall` fallan con `cudaErrorNoKernelImageForAvailable`, el backend local de `embeddings/reranking` intentó utilizar una GPU CUDA incompatible con los kernels disponibles.

Primero configura el runtime embedded de Hindsight para CPU:

```
hindsight-embed profile create hermes --merge --env  
HINDSIGHT_API_EMBEDDINGS_LOCAL_FORCE_CPU=true --env  
HINDSIGHT_API_RERANKER_LOCAL_FORCE_CPU=true
```

Si es necesario registrar las mismas variables en el Profile lois, ejecuta:

```
echo "HINDSIGHT_API_EMBEDDINGS_LOCAL_FORCE_CPU=true" >> ~/.hermes/profiles/lois/.env  
echo "HINDSIGHT_API_RERANKER_LOCAL_FORCE_CPU=true" >> ~/.hermes/profiles/lois/.env
```

Comprueba:

```
tail -n 10 ~/.hermes/profiles/lois/.env
```

Después cierra las sesiones de Hermes/Hindsight que estén en ejecución y abre nuevamente:

```
hermes -p lois chat
```

5.5 Confirmar que Hindsight está activo

```
hermes -p lois memory status
```

El estado debe indicar Hindsight como provider externo activo en el Profile lois.

5.6 Probar retain, recall y reflect

Abre el Profile lois:

```
hermes -p lois chat
```

En el chat, pide explícitamente a Hermes que utilice `hindsight_retain` y almacene un dato de prueba. Después finaliza la sesión, abre una nueva y pide que utilice `hindsight_recall` para recuperar la información.

Para probar síntesis entre memorias, almacena dos datos relacionados y pide que utilice `hindsight_reflect` para resumir las relaciones encontradas.

Nota: `retain` puede ser asíncrono; espera algunos segundos antes del primer `recall` cuando la memoria acaba de ser escrita.

5.7 Crear relaciones y recuperar conocimiento

Registra hechos relacionados, por ejemplo: Ana coordina el Curso Hermes; Bruno mantiene una API utilizada por el curso; el curso utiliza OpenViking para documentación.

Pide al agente que recupere las relaciones entre las entidades registradas.

Pide una reflexión que separe los hechos registrados de las inferencias.

Bloque 6 - Utilizar Codex en el mismo Profile alumno

No es necesario crear un Profile exclusivo para Codex. El runtime Codex puede activarse en el mismo Profile alumno utilizado con OpenViking. El Profile sigue siendo el mismo; solo cambia el runtime utilizado para ese tipo de sesión.

En el runtime `codex_app_server`, las herramientas que dependen directamente del loop interno de Hermes, como `delegate_task` y la herramienta `memory`, no están disponibles durante el turno. Después de la demostración de Codex, vuelve al runtime predeterminado antes de continuar con memoria o subagentes.

6.1 Instalar y autenticar Codex CLI

```
npm i -g @openai/codex
codex --version
codex login
```

Utiliza Codex CLI 0.130.0 o superior.

6.2 Preparar un Workspace para la prueba

```
mkdir -p ~/hermes-lab/aula3-codex
cd ~/hermes-lab/aula3-codex
pwd
```

Permanece en el directorio devuelto por `pwd`.

Inicia el Profile alumno desde este Workspace en el terminal.

Ejecuta `hermes -p alumno chat`.

6.3 Activar el runtime Codex

En el chat del Profile alumno:

```
/codex-runtime
/codex-runtime codex_app_server
```

Abre una nueva sesión después de activarlo.

6.4 Ejecutar una tarea de código

Pide a Hermes que confirme el directorio actual y enumere los archivos del Workspace.

Pide la creación de un pequeño proyecto Python con archivo principal, pruebas y README.

Sigue los cambios directamente en el directorio del Workspace mediante el terminal o el editor de archivos.

Ejecuta las pruebas desde el terminal solo si es necesario.

6.5 Volver al runtime predeterminado

/codex-runtime auto

Abre una nueva sesión antes de los siguientes bloques.

Bloque 7 - Subagentes

Los subagentes son ejecuciones temporales creadas mediante `delegate_task`. Reciben una tarea y un contexto aislado, realizan el trabajo y devuelven el resultado al agente principal. No son Profiles persistentes.

7.1 Demostrar un subagente

En el terminal, utiliza una sesión del Profile deseado iniciada con `hermes -p <profile> chat` y mantén el runtime predeterminado.

Pide al agente: “Delega a un subagente la tarea de comparar memoria persistente y contexto de conversación. Devuelve únicamente el resumen final”.

Sigue la ejecución mediante la interfaz de agentes cuando esté disponible.

Bloque 8 - Kanban: crear y acompañar un proyecto

Kanban organiza tareas duraderas en un tablero y registra el progreso del trabajo. En esta demostración, todas las etapas serán ejecutadas por el Profile alumno creado anteriormente.

8.1 Abrir Kanban

Abre **Dashboard > Kanban**.

Si la base de datos todavía no existe, el simple acceso a Kanban puede inicializarla automáticamente. Si es necesario inicializarla manualmente:

```
hermes kanban init
```

8.2 Crear el proyecto en Kanban

Proyecto: investigar una actualización técnica de Hermes, producir un resumen didáctico y revisar el resultado antes de finalizar.

Mantén **Orchestration** en **Manual** en la primera prueba.

Crea una tarea principal en **Triage** para representar el proyecto.

Crea las etapas del trabajo: investigar el tema, producir el material y revisar el resultado.

Define las dependencias entre las etapas.

Asigna todas las tareas al Profile alumno.

Comprueba el objetivo, el Workspace y los criterios de aceptación de cada tarea.

Mueve la primera tarea a **Ready**.

Utiliza **Nudge dispatcher** para ejecutar un ciclo inmediatamente cuando sea necesario.

Acompaña la tarea en **Ready, Running y Done** o **Blocked**.

Cuando una etapa finalice, acompaña la liberación de la siguiente tarea dependiente.

Repite el flujo hasta completar el proyecto.

Abre las tarjetas para visualizar historial, ejecución, comentarios y resultado.

8.3 Probar la orquestación automática

Después de confirmar el flujo manual, crea otra tarea principal en **Triage**.

Utiliza **Decompose** para generar subtareas.

Comprueba si las subtareas y dependencias tienen sentido y si el Profile alumno sigue siendo el responsable.

Solo después de esta comprobación, prueba **Orchestration: Auto**.

Bloque 9 - Cron y automatizaciones

Cron permite crear tareas únicas o recurrentes. Pueden crearse en lenguaje natural por el propio agente y supervisarse mediante Desktop o Dashboard.

9.1 Crear una automatización

Abre una conversación desde el terminal con el Profile elegido, por ejemplo: `hermes -p alumno chat`.

Pide: “Dentro de 5 minutos, crea un resumen breve del estado actual de este proyecto y entrégalo localmente”.

Abre el área **Cron** en Hermes Desktop o Dashboard.

Confirma nombre, programación, estado y destino.

Utiliza **Trigger now** para probar inmediatamente.

Prueba **Pause, Resume, Edit** y **Delete** mediante la interfaz.

Los cron jobs se ejecutan en sesiones nuevas. El prompt debe contener el contexto necesario para ejecutar la tarea de forma independiente.

Bloque 10 - A2A: comunicación entre agentes independientes

A2A es un protocolo de comunicación entre agentes independientes. Un Hermes puede llamar a otro Hermes como agente, descubrir sus capacidades y mantener una conversación A2A mediante `context_id`.

A2A no es obligatorio para Kanban. Para varios Profiles en la misma máquina, Kanban ya proporciona una cola de trabajo duradera. A2A es más apropiado cuando la comunicación debe atravesar procesos, máquinas o frameworks independientes.

10.1 Ejemplo: Profile principal llamando al Profile lois

El ejemplo utiliza el Profile principal como cliente A2A y el Profile lois como agente llamado. El mismo flujo puede reproducirse con otros nombres de Profile.

10.2 Habilitar A2A en el Profile lois

Ejecuta el setup del gateway del Profile lois y habilita A2A:

```
hermes -p lois gateway setup
```

Define un puerto local exclusivo para el A2A de lois, por ejemplo 9901.

Inicia explícitamente el gateway de lois desde el terminal:

```
hermes -p lois gateway run
```

10.3 Habilitar las herramientas A2A en el Profile principal

En el Profile principal, abre la configuración **Tools/Toolsets**.

Habilita el toolset a2a.

Registra el Profile lois como agente conocido apuntando al endpoint A2A local.

10.4 Llamar al agente lois

En el chat del Profile principal iniciado desde la línea de comandos, pide descubrir las capacidades del agente lois.

Envía una tarea breve a lois.

Haz una segunda pregunta manteniendo el mismo context_id para demostrar continuidad en la conversación entre agentes.

Consulta el historial de la conversación A2A cuando sea necesario.

Bloque 11 - Limpiar providers y eliminar Profiles

11.1 Desactivar Hindsight en el Profile lois

Antes de eliminar archivos locales, vuelve el Profile lois a la memoria integrada:

```
hermes -p lois memory off
```

Salida esperada: Memory provider: built-in only.

11.2 Eliminar componentes locales de Hindsight

Elimina el cliente instalado por el setup, cuando esté presente:

```
uv pip uninstall --python ~/.hermes/hermes-agent/venv/bin/python hindsight-client
```

Si hindsight-all está instalado en el mismo entorno, elimínalo también:

```
uv pip uninstall --python ~/.hermes/hermes-agent/venv/bin/python hindsight-all
```

Si el comando informa que hindsight-all no está instalado, continúa normalmente.

Elimina únicamente la instancia PostgreSQL embedded creada por Hindsight:

```
rm -rf ~/.pg0/instances/hindsight-embed-hermes
```

Elimina las variables HINDSIGHT_* del Profile lois:

```
sed -i '/^HINDSIGHT_/d' ~/.hermes/profiles/lois/.env
```

Comprueba que no queden variables:

```
grep HINDSIGHT ~/.hermes/profiles/lois/.env
```

11.3 Eliminar OpenViking instalado mediante uv

uv tool uninstall openviking

Confirma que ya no aparece en la lista:

```
uv tool list
```

Para borrar también la configuración y los datos locales de OpenViking:

```
rm -rf ~/.openviking
```

Para borrar la caché/modelo local descargado por OpenViking cuando quieras restablecer completamente el entorno:

```
rm -rf ~/.cache/openviking
```

11.4 Eliminar Profiles

El comando siguiente elimina permanentemente el Profile indicado, incluida su configuración, memoria, sesiones y skills. El Profile default no debe eliminarse.

```
hermes profile delete <profile-name> --yes
```

Ejemplos utilizados en esta clase:

```
hermes profile delete aluno --yes
```

```
hermes profile delete lois --yes
```

Confirma mediante la CLI:

```
hermes profile list
```

Si aluno o lois ya no aparece en la lista, la eliminación se completó. Si Dashboard todavía muestra una entrada stopped, recarga o reinicia Dashboard para actualizar la interfaz.

Clase 4

Acceso remoto y múltiples usuarios, creación y uso de Skills e integración con servidores MCP.

Referencias oficiales

Profiles: <https://hermes-agent.nousresearch.com/docs/user-guide/profiles/>

Profile Commands: <https://hermes-agent.nousresearch.com/docs/reference/profile-commands/>

Issue Hermes Desktop cwd #38855: <https://github.com/NousResearch/hermes-agent/issues/38855>

Issue Hermes Desktop profile/cwd #79406:
<https://github.com/NousResearch/hermes-agent/issues/79406>

Issue de Profiles locales de Hermes #81492:
<https://github.com/NousResearch/hermes-agent/issues/81492>

Memory: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory/>

Memory Providers: <https://hermes-agent.nousresearch.com/docs/user-guide/features/memory-providers/>

OpenViking Quick Start - Server Mode:
<https://github.com/volcengine/OpenViking/blob/main/docs/en/getting-started/03-quickstart-server.md>

Codex App-Server Runtime: <https://hermes-agent.nousresearch.com/docs/user-guide/features/codex-app-server-runtime>

Subagent Delegation: <https://hermes-agent.nousresearch.com/docs/user-guide/features/delegation/>

Kanban: <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban>

Kanban Tutorial: <https://hermes-agent.nousresearch.com/docs/user-guide/features/kanban-tutorial>

Cron: <https://hermes-agent.nousresearch.com/docs/user-guide/features/cron/>

A2A: <https://hermes-agent.nousresearch.com/docs/user-guide/messaging/a2a>

Hindsight plugin: <https://github.com/NousResearch/hermes-agent/blob/main/plugins/memory/hindsight/README.md>

Ejemplo Hindsight FORCE_CPU: <https://github.com/NousResearch/hermes-agent/issues/8972>

Variables de entorno de Hindsight: <https://github.com/vectorize-io/hindsight/blob/main/.env.example>

OpenViking Quick Start: <https://github.com/volcengine/OpenViking/blob/main/docs/en/getting-started/03-quickstart-server.md>

Contenido traducido del portugués al español mediante IA.